

PROJECT REPORT

Smart Surveillance and Motion Analysis using Computer Vision

Submitted by

Student Name: Vedansh Talaria

Registration Number: 24BAI10419

Course: Computer Vision

Institution: VIT Bhopal University

Repository: _____

This report documents the design, implementation, testing and limitations of a terminal-based computer vision pipeline for recorded surveillance video.

1. Introduction

Manual review of surveillance footage requires an operator to inspect many consecutive frames. This project demonstrates a modular computer vision workflow that converts a recorded video into object tracks, motion measurements and structured event records. The implementation combines classical image-processing operations with a pretrained YOLO detector/tracker.

The pipeline is deliberately command-line based. It can be executed from a terminal, accepts runtime parameters, writes machine-readable outputs, and does not require a GUI for project execution.

Computer Vision concepts applied:

- Image resizing and aspect-ratio preservation.
- Gaussian smoothing, a convolution-based denoising operation.
- CLAHE contrast enhancement and grayscale conversion.
- Canny edge extraction.
- Frame differencing and binary motion masks.
- Farneback dense optical flow for motion magnitude.
- Object detection, confidence scoring and persistent multi-object tracking.

2. Problem Statement and Objectives

A recorded surveillance video can contain many frames in which only a small subset contains meaningful motion. Manually locating these events is repetitive. The project therefore provides an automated pipeline that detects and tracks visible objects, estimates motion between frames, and records motion-associated observations for later review.

Objectives

- Read and validate a stored surveillance video.
- Preprocess frames using resizing, Gaussian denoising, CLAHE and Canny edge extraction.
- Detect objects and maintain persistent track IDs using YOLO tracking.
- Estimate motion using both changed-pixel masks and dense optical flow.
- Reduce duplicate event records with a configurable per-track cooldown.
- Generate an annotated MP4, events.csv and summary.json.
- Provide automated tests for deterministic modules and clear command-line documentation.

2.1 Scope

In scope: recorded video input, frame preprocessing, object detection/tracking, motion estimation, event logging, analytics and terminal execution. Out of scope: facial recognition, identity inference, automatic suspicious-activity judgment, live CCTV deployment, cloud deployment and training a new model.

3. Functional Requirements

ID	Requirement
FR-1	Accept a compatible video path through the command line.
FR-2	Validate video availability and readable properties.

FR-3	Resize frames while preserving aspect ratio.
FR-4	Apply Gaussian denoising, CLAHE and Canny edge extraction.
FR-5	Detect and track objects using a pretrained YOLO model.
FR-6	Estimate motion using frame differencing and dense optical flow.
FR-7	Log motion-associated tracked-object events with motion metrics.
FR-8	Generate an annotated output video unless disabled.
FR-9	Generate CSV and JSON analytics.
FR-10	Expose processing parameters through command-line options.

3.1 Non-Functional Requirements

Requirement	Implementation
Performance	Configurable processing width, frame limit and optional video writing.
Reliability	Input validation and resource release through structured cleanup.
Maintainability	Separate modules with centralized defaults and CLI overrides.
Usability	Terminal execution and a documented --help interface.
Resource efficiency	Frames are resized before expensive analysis.
Testability	pytest coverage for deterministic modules.
Observability	Console summary plus structured CSV/JSON outputs.

4. System Architecture

The application follows a single-responsibility pipeline. `main.py` orchestrates the modules while each module performs one major task. This structure supports replacement and independent testing.

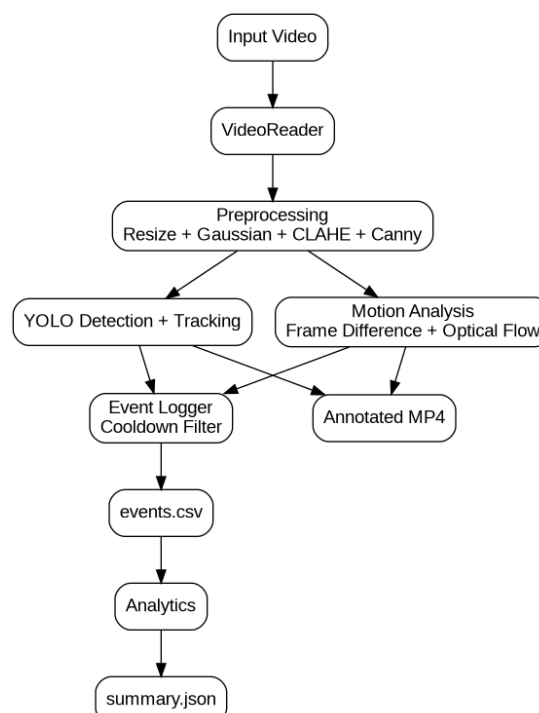


Figure 1. System architecture.

Pipeline: VideoReader → ImageProcessor → YOLO Tracking and Motion Analysis → Cooldown Filter → Event Logger → Analytics. Tracking and motion results are also used to create the annotated video.

5. Design Diagrams

5.1 Use Case Diagram

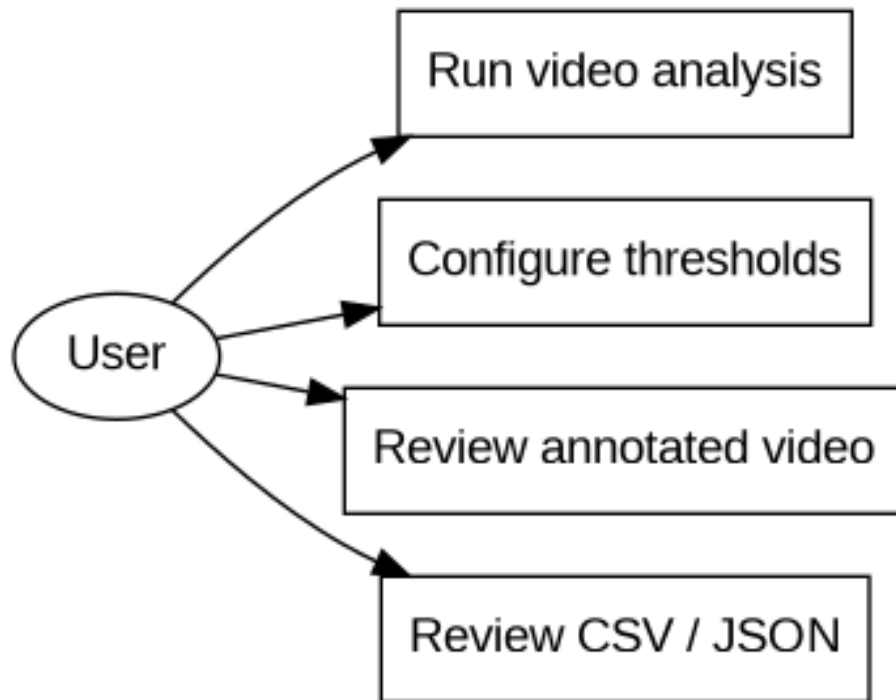


Figure 2. User interactions with the system.

5.2 Sequence Diagram

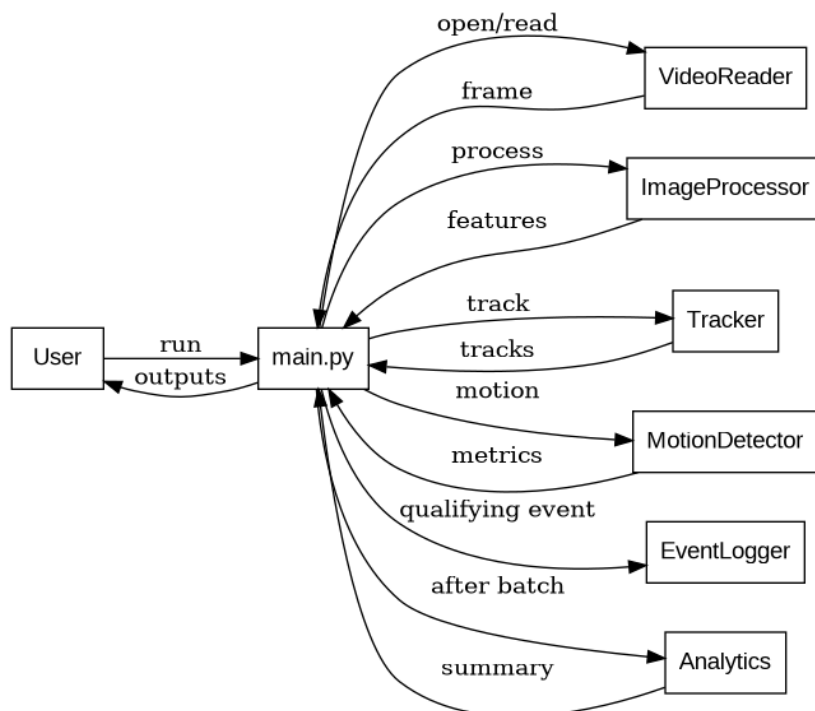


Figure 3. Sequence of a processing run.

5.3 Component Diagram

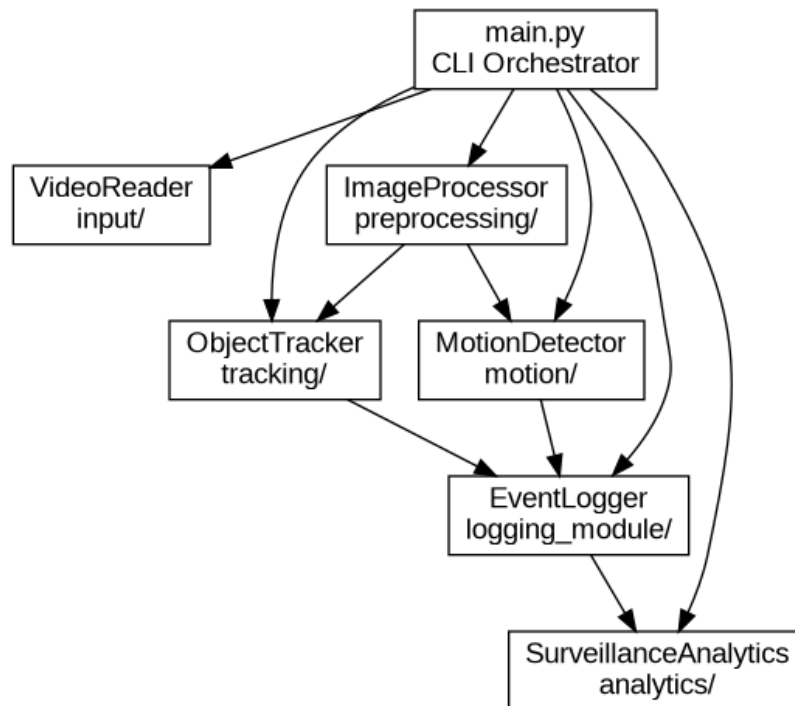


Figure 4. Component/module relationships.

6. Implementation Details

Technology stack

Python 3.11+, OpenCV, NumPy, Ultralytics YOLO, pytest, Git and GitHub.

Preprocessing

Frames are resized to a configurable width. GaussianBlur reduces high-frequency noise. CLAHE improves local contrast and Canny produces an edge map. The resized color frame is retained for YOLO tracking so that preprocessing features do not discard color information required by the detector.

Detection and tracking

ObjectTracker loads the supplied YOLO model and calls persistent tracking. Each returned track contains a tracker ID, class ID, class name, confidence and bounding box. ObjectDetector is retained as a reusable standalone detection component even though the main pipeline uses the tracking path.

Motion analysis

The MotionDetector converts frames to blurred grayscale images. Absolute frame differences are thresholded and morphologically cleaned to obtain a changed-pixel mask. Farneback dense optical flow is then used to calculate a mean motion magnitude. Motion is flagged when either the changed-pixel count exceeds its threshold or the mean flow exceeds the configured minimum.

Event logging and analytics

When motion is present, a per-track cooldown prevents an object from generating an event on every consecutive frame. CSV records include timestamp, frame number, event type, object ID, class,

confidence, changed pixels and mean flow. Analytics aggregates total events, unique IDs, class counts and averages into summary.json.

7. Storage Design and Design Decisions

Storage schema

Field	Meaning
timestamp	Event creation time
frame_number	Frame in which the event was recorded
event_type	Currently Motion Detected
object_id	Persistent tracker ID
object_class	YOLO class name
confidence	Detection/tracking confidence
motion_pixels	Changed pixels in the motion mask
mean_flow	Mean optical-flow magnitude

Design decisions

- Command line: avoids GUI-only execution and makes evaluation reproducible.
- YOLO tracking: supplies object classes, confidence values and persistent IDs.
- Dual motion measurement: frame differencing is interpretable while optical flow adds motion magnitude.
- Cooldown: reduces repetitive event records without removing the underlying tracked object.
- CSV + JSON: lightweight and portable for a batch academic project.
- Lazy model import: allows deterministic tests and --help to run without model inference.

8. Installation and Execution

The repository is designed to be run from a terminal.

Environment setup

```
python -m venv .venv
```

```
Windows: .\.venv\Scripts\Activate.ps1
```

```
Linux/macOS: source .venv/bin/activate
```

```
python -m pip install -r requirements.txt
```

Run

```
python main.py --help
```

```
python main.py --input input/test_video.mp4 --output-dir output --max-frames 30
```

```
python main.py --input input/test_video.mp4 --output-dir output
```

```
python main.py --input input/test_video.mp4 --output-dir output --no-video
```

Generated files

- **output/surveillance_output.mp4** — annotated video.
- **output/events.csv** — structured motion/object events.
- **output/summary.json** — run-level analytics.

The included repository contains the YOLO model file and sample video. Exact inference time and event counts depend on the execution environment, model version, thresholds and input video; they should be measured during the student's own run.